Today
Welcome to your next lesson: Analyzing stack and queue operations.

I'm preparing your personalized lesson based on what you've learned, what's coming up, and the best ways to help you stay strong on these new topics.

Step by step, you're moving forward.

Imagine two situations where speed matters:

A stack of plates in a cafeteria: you add and remove plates from the top only.
A line of people at a ticket booth: new people join at the back, and the person served leaves from the front.

For this first step, focus just on the stack (the plates).

In Python, a stack is usually just a list where:

stack.append(x) = put a plate on top (push)
stack.pop() = take a plate from the top (pop)

Both of these work on one end only (the right end of the list), and they each take constant time — written as O(1) — no matter how many items are in the stack.

Does that picture of a stack (plates + append/pop) make sense so far, or is there any part you want me to slow down and rephrase?

Yes, that makes sense

What do you think the answer is in the question below? 👇

You have this Python code:

```
stack = []
stack.append(10)
stack.append(20)
stack.append(30)
value = stack.pop()
```

After the pop(), which number is removed from the stack, and what is the time complexity of that pop() operation?

A.

10 is removed, time complexity O(n)

B.

10 is removed, time complexity O(1)

C.

30 is removed, time complexity O(n)

D.

30 is removed, time complexity O(1)

Nice work, Ron 🎯

You were exactly right:

The stack is built as: [10, 20, 30]
pop() removes from the top/right end, so it removes 30
That pop() touches only that one element at the end, so its time stays the same no matter how big the stack is → O(1)
Now let's switch to the queue / line of people idea next:

in a queue, we add at the back and remove from the front.

Are you ready to look at how we model a queue in Python and why some ways are slower than others? (yes/no)

Yes, let's continue

Great 👌

For a queue, think "people in line":

New person joins at the back → queue.append(x)
Person served leaves from the front → something like queue.pop(0)

Here's the key idea:

With a normal Python list, doing pop(0) (front of the list) is slow because every other element has to shift left one position. That makes it O(n) time.

append at the end is still O(1), but pop(0) from the front is O(n).

Before we try code, is this mental picture clear?

Back of line: cheap (append, O(1))
Front of line with pop(0): expensive (O(n) because of shifting)

Yes / kind of / not really?

Yes, that's clear

Here's a visual showing two queues side by side: one using a list with pop(0) and one using a deque with popleft() 👇

Try clicking enqueue/dequeue a few times and watch:

how the list queue has to slide everything left on dequeue (O(n))
while the deque queue just removes the front box with no shifting (O(1))
After playing with it:

Does the difference between list pop(0) = O(n) and deque.popleft() = O(1) feel clear?

List queue vs deque queue operations

Yes, it feels clear

Nice, you're in a good spot here 👍

Let's now connect this to choosing the right structure for a problem.

Very simple rule of thumb:

If your logic is “last in, first out” (like undo history, browser back button, plate stack) → use a list as a stack with append / pop() → both ends at the right, both O(1).
If your logic is “first in, first out” and you remove from the front a lot (like a line of tasks to process) → use a collections.deque as a queue with append / popleft() → both O(1).
Let me check this with you using a concrete scenario:

You’re building a task system where:

New tasks are added at the back
The oldest task is always processed first
You remove from the front very often

Which would you pick in Python for this queue: a list with pop(0) or a deque with popleft()? Just answer in your own words.

list with pop(0)

Good, this is exactly the kind of decision I want you to think through 👍

For that task system (oldest task processed first, lots of removals from the front), a deque with popleft() is the better choice, because:

List pop(0) has to shift all the remaining items left → O(n) each time.
deque.popleft() just removes the front item directly → O(1) each time.

So for true first-in-first-out with many front removals, deque wins.

Let’s lock this in with a small check:

If you instead needed an undo history (last action undone first), which structure and operations would you use in Python?

You can answer in a short phrase, like:

“list with … and …”

list with append / pop()

Exactly right 🔥

Undo history is last-in, first-out
A list with append() to add actions and pop() to undo the last one is perfect
Both operations are O(1), so it stays fast even with many actions
You’ve now seen:

Stacks → list with append / pop() → LIFO, O(1) each
Queues → deque with append / popleft() → FIFO, O(1) each
Why list.pop(0) is slow → O(n) because of shifting

Want to do a quick multiple-choice question to check everything is solid?

Yes, give me the quiz

What do you think is the best choice here? 👇

You’re implementing a download manager in Python:

New download jobs are added to the back of a queue.

The oldest job is always processed first.
There are a lot of removals from the front of the queue.
Which data structure and operations give you the most efficient behavior for this queue?

A.

A Python list with append() to add and pop() (no index) to remove.

B.

A Python list with append() to add and pop(0) to remove.

C.

A collections.deque with append() to add and popleft() to remove.

D.

A collections.deque with appendleft() to add and pop() to remove.

Nice, Ron — you nailed it 🎯

A collections.deque with append() + popleft() is exactly what you want because:

It matches the behavior: add at back, remove from front (true FIFO).
Both append() and popleft() on a deque are O(1), so even with tons of downloads, it stays fast.
The list version with pop(0) would be O(n) on each front removal because of shifting.
At this point, you've:

Identified time complexity of stack append / pop() ☑
Understood why list pop(0) is slow and deque.popleft() is fast ☑
Chosen efficient structures for LIFO vs FIFO based on cost ☑

You've mastered this lesson's goals 🎓

Do you feel ready to wrap up this lesson and move to the next one, or do you want one more practice question first?

Move to the next lesson

Awesome work today, Ron 🎓

Quick recap of what you now own:

Stacks:

Implemented with a Python list using append() and pop()
Both are O(1) → perfect for last-in, first-out (undo, back button, etc.)
Queues:

List + pop(0) is O(n) because every front removal shifts all later elements
collections.deque with append() + popleft() is O(1) for both ends → ideal for first-in, first-out task/job queues
Choosing structures:

LIFO → list stack
FIFO with many front removals → deque queue
You're ready for the next lesson 🚀
When you're set, hit Next Lesson in the Learn tab, and I'll be right there with you.